European Journal of Operational Research 286 (2020) 867–882

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

A MIP model and a biased random-key genetic algorithm based approach for a two-dimensional cutting problem with defects☆

José Fernando Gonçalves[a,*], Gerhard Wäscher[b,c]

[a] LIAAD, INESC TEC, Faculdade de Economia, Universidade do Porto Rua Dr. Roberto Frias, s/n, 4200-464 Porto, Portugal
[b] Faculty of Economics and Management, Otto-von-Guericke-Universität Magdeburg, Germany
[c] School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, China

ARTICLE INFO

Article history:
Received 21 September 2018
Accepted 11 April 2020
Available online 28 April 2020

Keywords:
Cutting
Packing
Two-dimensional
Non-guillotine
defects
BRKGA
Biased random-key genetic algorithm

ABSTRACT

This paper addresses a two-dimensional (*2D*) non-guillotine cutting problem, where a set of small rectangular items of given types has to be cut from a large rectangular stock plate having defective regions so as to maximize the total value of the rectangles cut. The number of small items of each item type which can be cut from the large object is unrestricted. A novel MIP model and a hybrid approach combining a novel placement procedure with a biased random-key genetic algorithm (*BRKGA*) are presented. The parameters used by the novel placement procedure for the development of a cutting plan are evolved by the *BRKGA*. The management of the free spaces and of the defects uses a maximal-space representation. The approach is evaluated and compared to other approaches by means of a series of detailed numerical experiments using 5414 benchmark instances taken from the literature. The experimental results validate the quality of the solutions and the effectiveness of the proposed algorithm.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

In this paper, we consider a two-dimensional cutting problem in which small rectangular items of given types have to be cut from a single large rectangular stock plate which contains defective regions. Not all small items can be accommodated by the non-defective regions of the stock plate, thus the total value of the small items to be cut has to be maximized. The problem is relevant both from a theoretical and a practical point of view because it arises in various production processes. The presence of defects can be encountered in many cutting problems. Steel coils produced in the steel industry may contain contaminated areas, which cannot be used since they do not meet the customers' quality requirements. Natural products like leather often possess areas of inferior quality or holes from which no parts suitable for the production of, e.g., leather jackets or shoes, can be obtained. Wooden boards in the furniture industry may include knotholes, which cannot be used for cutting furniture parts.

This work is motivated by the introduction of cutting processes that make use of non-guillotine cuts. Guillotine cutting processes are very common in industry. However, because of increased market competition, any small improvement in the efficiency of the cutting process is relevant. By being more flexible, non-guillotine cutting processes have the potential to improve efficiency. They, however, also possess a much larger search space and, thus, are more difficult to solve. The goal of this paper is to determine how much improvement in cutting efficiency can be achieved by using a non-guillotine cutting process as opposed to a guillotine process.

In this paper, we address a specific version of the problem where each defective region can be represented by means of a rectangle or a set of rectangles. However, we do not impose any additional constraints w.r.t. the cutting process itself (i.e. guillotine constraints). For this problem, we introduce a novel mixed integer programming (MIP) model and a hybrid heuristic solution approach. The hybrid approach combines a random-key based genetic algorithm with a novel placement strategy.

The remainder of the paper is organized as follows. Section 2 provides a thorough definition of the problem under discussion while Section 3 presents a review of the related literature. The MIP model for the representation of the problem is presented in section 4. Section 5 introduces the solution approach, describing in detail the random-key based genetic algorithm and the placement strategy. In Section 6, we report on computational experiments which have been carried out in order to evaluate the

☆ This work has been supported by project NORTE-01-0145-FEDER-000020 financed by the North Portugal Regional Operational Programme (NORTE 2020), under the PORTUGAL 2020 Partnership Agreement, and through the European Regional Development Fund (ERDF).

* Corresponding author.
E-mail addresses: jfgoncal@fep.up.pt (J.F. Gonçalves), gerhard.waescher@ovgu.de (G. Wäscher).

https://doi.org/10.1016/j.ejor.2020.04.028
0377-2217/© 2020 Elsevier B.V. All rights reserved.

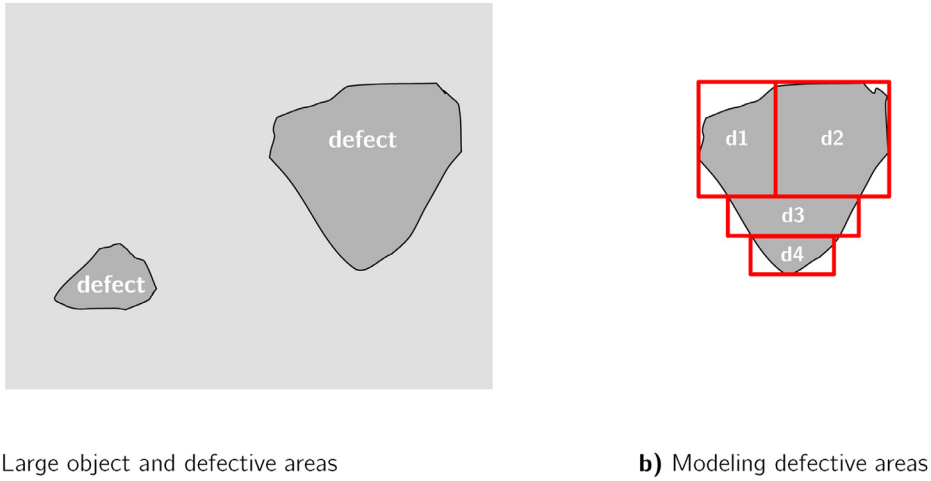


Fig. 1. The problem.

proposed solution approach. The paper concludes with a summary and an outlook on future research opportunities in Section 8.

2. Problem statement

Let O be a single rectangular large object with dimensions (W, H) along the horizontal X-axis and the vertical Y-axis, respectively. O contains (a small number of) defective regions ("defects" in short) which are approximated by a set of non-overlapping rectangles, $d = 1, 2, \ldots, D$ (see Fig. 1). Each rectangle d is defined by its centroid (X_d, Y_d) and integer dimensions (W_d, H_d) along the horizontal and the vertical axis, having an area equal to $A_d = W_d \times H_d$. W_d and H_d represent the width and the height of defect d, respectively.

We point out that the rectangles used to cover each of the defects are an input to the problem. In practice, they are determined by experienced users using CAD systems, taking into consideration the material type (metal, wood, fabric, etc.) and the cutting process (laser, water, etc.). In order to represent a defect with an acceptable precision a small number of rectangles, usually less than four, is required.

Let T be the set of small rectangular item types that can be cut from O, where each item type $t = 1, \ldots, T$ is defined by its dimensions (W^t, H^t) along the horizontal and vertical axis, i.e. covering an area equal to $A^t = W^t \times H^t$, and a having a value V^t. Without loss of generality it is assumed that the width W^t and height H^t of the small item types are positive integers. The demand for each small item type t is unlimited.

From each item type t a number of copies (i.e. small items) are to be selected and laid out on the large object. The edges of the small items must be arranged in parallel to the edges of the large object (orthogonal layout) and the following conditions must be satisfied:

- all the small items must lie entirely within the large object O;
- and the small items must not overlap each other or the defective regions.

For each small item type t the orientation according to which it has to be laid out on the large object is fixed. However, a rotation of 90° can be considered by representing the respective item type t by two item types, namely by one of dimensions (W^t, H^t) and the other with dimensions (H^t, W^t).

The problem now consists of determining how many items of each type t should be assigned to and how they are to be laid on the large object such that the total value of the assigned small items is maximized. A solution to the problem can be represented by the coordinates of the centroid (x_i, y_i) and the horizontal (w_i) and vertical (h_i) dimensions of each small single item i that has been assigned to the large object.

According to the typology of Wäscher, Haußner, and Schumann (2007), the underlying problem can be categorized as a two-dimensional single large object placement problem (*2D_rect_SLOPP_D*). The presence of the a specific feature, namely the defective large object, constitutes a variant of this problem type.

Garey and Johnson (1979) have shown that the 2D_rect_SLOPP is NP-hard. Since the inclusion of a defective large object represents a generalization, we conclude that the problem considered here, the 2D_rect_SLOPP with a defective large object, is also NP-hard.

3. Literature review

The two-dimensional (non-guillotine constrained) cutting problem described above has been studied by different authors, however without considering defects on the large object. Exact solution approaches have been proposed by several authors. Beasley (1985) presented an upper bound derived from a Lagrangian relaxation of a 0–1 integer linear programming formulation and a branch and bound algorithm. Tsai, Malstrom, and Meeks (1988) and Scheithauer and Terno (1993) provided integer programming formulations for the problem. In the latter publication binary variables are used in order to decide whether an item is cut to the right/left or above/below of another item. Hadjiconstantinou and Christofides (1995) introduced a branch and bound algorithm in which the search is limited by using an upper bound based on a Lagrangian relaxation procedure; subgradient optimization is used to improve the bound. Arenales and Morabito (1995) proposed an approach based on an AND/OR-graph together with a branch and bound search. Healy, Creavin, and Kuusik (1999) developed an algorithm based on the identification of the empty spaces from which further items can be cut. Boschetti, Hadjiconstantinou, and Mingozzi (2002) introduced new upper bounds derived from different relaxations of a new integer programming formulation of the constrained two-dimensional non-guillotine cutting problem.

Heuristic and meta-heuristics procedures have also been proposed by several authors. Lai and Chan (1997b) developed a heuristic based on simulated annealing which encodes the order in which pieces should be cut. The same authors (Lai & Chan, 1997a) presented a heuristic based on an evolutionary strategy approach

which includes an improvement procedure based on dividing the ordered list into active (cut) and inactive (uncut) items and seeing if any (currently) uncut items can be cut. Their mutation process involves swapping two items in the ordered list. Leung, Chan, and Troutt (2003) created a hybrid heuristic combining simulated annealing with a genetic algorithm. Beasley (2004) proposed a genetic algorithm based on a nonlinear formulation of the problem. Alvarez-Valdes, Parreño, and Tamarit (2005) developed a heuristic based on a greedy randomized adaptive search procedure (GRASP) where the constructive phase explicitly considers the possibility of simultaneously cutting several items of the same type and forming a block. Alvarez-Valdes, Parreño, and Tamarit (2007) introduced a tabu search algorithm which includes several moves based on reducing and inserting blocks of pieces and includes diversification and intensification procedures based on long-term memory.

Despite the large body of literature in the field of cutting and packing (C&P), the number of studies concerning cutting problems with defective large objects is rather limited also see Afsharian (2013) for an in-depth study of the existing literature related to cutting problems with defective large objects). Gilmore and Gomory (1965) discussed the Single Stock Size Cutting Stock Problem in one and two dimensions. Hahn (1968) addressed a problem variant where the (single) large object possesses multiple defects and the value function for the item types is non-linear; the layout of the solution must be guillotineable and the number of stages of the guillotine cuts is constrained. Beasley (2004) considered a slightly different problem which is characterized by additional constraints on the availability of the small items of each type. At first, the author formulates a non-linear model of the problem without defects before he shows how this model can be modified for dealing with a defective large object. Carnieri, Mendoza, and Luppold (1993) introduced the two-dimensional stage-unconstrained, guillotineable layout single large object cutting problem into the literature. However, they only allowed for a single defect in the large object. They developed a dynamic programming-based heuristic which extends an algorithm suggested by Gilmore and Gomory (1961). Vianna and Arenales (2006) proposed a branch-andbound algorithm which is based on an AND/OR-graph approach. Neidlein and Wäscher (2008) proposed an improved bound which can be used within these algorithms. Afsharian, Niknejad, and Wäscher (2014) developed a heuristic, dynamic programming-based approach for a two-dimensional guillotinable layout cutting problem with a defective large object. In order to improve the computational efficiency and to reduce the storage space requirements the authors revisit the discretization sets proposed by Herz (1972).

4. Model formulation

The model formulation of the above-stated 2D_rect_SLOPP with a defective large object presented in this section is based on the concept of a "generic" small item. This is an item of which we do not know in advance whether it will be used and of which item type it will consist of. Using this concept of a generic item, the problem can be formulated in an intuitive way in the form of a mixed integer programming model for which the following additional notation is introduced:

- N^t : Maximum number of small items of type t that can be assigned to the large object. $\lfloor N^t = (W \times H - \sum_d A_d)/A^t \rfloor$, where $\lfloor x \rfloor$ corresponds to the largest integer smaller than or equal to x;
- N : Maximum number of generic small items of any type that can be assigned to the large object. $N = \max_t \{N^t\}$;
- I_i^t : Binary variable, equal to 1 if the generic item i is assigned to the large object and is cast into a small item of type t, 0 otherwise;
- $P_{i,j}$, $Q_{i,j}$: Binary variables used for modeling the non-overlapping constraints between small items i and j. If $P_{i,j} = 0$ and $Q_{i,j} = 0$ then i and j are separated horizontally with i on the right of j. If $P_{i,j} = 1$ and $Q_{i,j} = 0$ then i and j are separated horizontally with i on the left of j. If $P_{i,j} = 0$ and $Q_{i,j} = 1$ then i and j are separated vertically with i above j. If $P_{i,j} = 1$ and $Q_{i,j} = 1$ then i and j are separated vertically with with i below j;
- $R_{i,d}$, $S_{i,d}$: Binary variables used for modeling the non-overlapping constraints between small items i and defects d.

Then, the problem can be formulated as follows (2*DMIP*):

$$Maximize\ value = \sum_{i=1}^{N} \sum_{t=1}^{T} V^t I_i^t \tag{1}$$

Subject to:

Generic small items i and j**must not overlap**

$$x_i - x_j + W\left(P_{i,j} + Q_{i,j}\right) \geq \frac{w_i + w_j}{2} \qquad \forall i, j \mid j > i \tag{2}$$

$$x_j - x_i + W\left(1 - P_{i,j} + Q_{i,j}\right) \geq \frac{w_i + w_j}{2} \qquad \forall i, j \mid j > i \tag{3}$$

$$y_i - y_j + H\left(1 + P_{i,j} - Q_{i,j}\right) \geq \frac{h_i + h_j}{2} \qquad \forall i, j \mid j > i \tag{4}$$

$$y_j - y_i + H\left(2 - P_{i,j} - Q_{i,j}\right) \geq \frac{h_i + h_j}{2} \qquad \forall i, j \mid j > i \tag{5}$$

Generic small items i and defects d**must not overlap**

$$x_i - X_d + W\left(R_{i,j} + S_{i,j}\right) \geq \frac{w_i + W_d}{2} \qquad \forall i, d \tag{6}$$

$$X_d - x_i + W\left(1 - R_{i,j} + S_{i,j}\right) \geq \frac{w_i + W_d}{2} \qquad \forall i, d \tag{7}$$

$$y_i - Y_d + H\left(1 + R_{i,j} - S_{i,j}\right) \geq \frac{h_i + H_d}{2} \qquad \forall i, d \tag{8}$$

$$Y_d - y_i + H\left(2 - R_{i,j} - S_{i,j}\right) \geq \frac{h_i + H_d}{2} \qquad \forall i, d \tag{9}$$

Each generic small items must lie entirely inside the large object *O*

$$\frac{w_i}{2} \leq x_i \leq W - \frac{w_i}{2} \qquad \forall i \tag{10}$$

$$\frac{h_i}{2} \leq y_i \leq H - \frac{h_i}{2} \qquad \forall i \tag{11}$$

Each generic small item i**can be converted into at most one small item type**

$$\sum_{t=1}^{T} I_i^t \leq 1 \qquad \forall i \tag{12}$$

Determination the width of generic small item *i*

$$w_i = \sum_{t=1}^{T} I_i^t W^t \qquad \forall i \tag{13}$$

Determination the height of generic small item *i*

$$h_i = \sum_{t=1}^{T} I_i^t H^t \qquad \forall i \tag{14}$$

At most N^t generic small items can be cast into small item of type t

$$\sum_{i=1}^{N} I_i^t \leq N^t \qquad \forall t \tag{15}$$

Variable domain constraints

$$x_i, y_i, w_i, h_i \geq 0 \;\; \forall i \quad \text{and} \quad I_i^t,\, P_{i,j},\, Q_{i,j} \in \{0, 1\} \tag{16}$$

The objective function (1) maximizes the total value of the small items assigned to the large object. Two groups of constraints can be identified. The first group (2–11) refers to the generic small items i and j and includes the constraints associated with:

- the non-overlapping between the generic small items i and j (2)–(5);
- the non-overlapping between each generic small item i and all defective areas d (6)–(9);
- the containment conditions that impose that all the generic small items i must lie entirely within the large object O (10)–(11).

The second group of constraints (12)–(15) is responsible for converting the generic small items into one particular small item type. Constraints (12) impose that a generic small item can be converted into at most one particular small item type. Constraints (13)–(14) define the width and the height of each generic item i according to the small item type that it was cast into. Constraints (15) limits the number of generic items that can be cast into each small item type t. Finally, constraints (16) are the domain constraints for the variables.

Andrade, Birgin, Morabito, and Ronconi (2014) present a MIP for two-dimensional non-guillotine cutting problem with usable leftovers which has similar types of constraints. However, they do not include defects or use the concept of a generic item which reduces significantly the number of non-overlapping constraints.

The proposed mixed integer model 2*DMIP* will be used in our numerical experiments for providing optimal solutions for small problem instances. It fails to provide optimal or even near-optimal solutions for larger, more realistic instances due to the large number of binary variables. In order to overcome this deficiency we developed a new solution methodology, which combines a biased random-key genetic algorithm with a novel placement strategy. In the next section we describe the new methodology.

5. New approach

We begin this section with an overview of the proposed solution methodology. This is followed by a discussion of the biased random-key genetic algorithm, including detailed descriptions of the solution encoding and decoding, evolutionary process and novel placement strategy.

5.1. Overview

The new approach consists of a constructive heuristic algorithm which places, one at a time, each small item on the large rectangular object O. To avoid overlapping between the small items we propose a novel placement strategy which combines the concept of *empty maximal-spaces,* as described in Lai and Chan (1997b), and a novel, very efficient placement procedure which selects one of two placement heuristics for positioning the small items within the set of possible empty maximal spaces. The new approach combines a biased random-key genetic algorithm and the novel placement strategy.

The genetic algorithm serves as an instrument for evolving the encoded parameters, i.e. the chromosomes, that are representing the small item type placement sequence (*TPS*), the vector of placement heuristics (*VPH*), and the vector of type quantities (*VTQ*). The vectors of encoded parameters *TPS*, *VPH*, and *VTQ* are transformed into a solution using a decoding procedure. The following phases are applied to decode each chromosome:

1. *Type placement sequence decoder*: the first phase decodes part of the chromosome into the *TPS*, i.e., the sequence in which the small item types are placed on the large object;
2. *Placement heuristic decoder*: the second phase decodes part of the chromosome into the *VPH*, i.e., the vector of heuristic placement procedures;
3. *Type quantity decoder*: the third phase decodes part of the chromosome into the *VTQ*, i.e., vector of parameters that will be used by the decoder to determine the quantities that will be cut for each small type in *TPS*.
4. *Placement strategy*: the fourth phase makes use of *TPS*, *VPH*, and *VTQ*, defined in phases 1, 2, and 3, and places all the small items on the large object O using the novel placement strategy.
5. *Fitness evaluation*: the final phase computes the fitness of the solution obtained in phase 4 (a measure of quality of the cutting) using Eq. (1).

Fig. 2 illustrates the sequence of steps applied to each chromosome generated by the *BRKGA*.

The remainder of this section describes the genetic algorithm, the decoding procedure, and the placement strategy in detail.

5.2. Biased random-key genetic algorithms

Random-key genetic algorithms (*RKGA*) for solving sequencing problems were introduced in Bean (1994). In a *RKGA*, chromosomes are represented as vectors of randomly-generated real numbers in the interval [0,1]. A *decoder* is a deterministic algorithm which associates a solution and a objective value or *fitness* to the input chromosome of a combinatorial optimization problem.

The *population, P*, of random-key vectors of an *RKGA* evolves over a number of *generations* (iterations). The initial population, P_0, is made up of p vectors of r random keys. Each component, or random key, of a solution vector is generated independently at random in the real interval [0,1]. After the fitness of each individual is computed by the decoder in generation g, the population is partitioned into two groups of individuals: a small group of e *elite* individuals, i.e. those with the best fitness values, and the remaining set of $p - e$ *non-elite* individuals. To evolve the population of generation g, a new generation $(g + 1)$ of individuals is produced. All elite individuals of the population of generation g are copied to the population of generation $g + 1$. *RKGAs* implement mutation by introducing *mutants* into the population. A mutant is a vector of random keys generated in the same way that an element of the initial population is generated. Its role is similar to that of mutation in other genetic algorithms (Goldberg, 1989), i.e. it is meant to introduce diversification into the population and avoid an early convergence of the entire population to a local optimum. At each generation, a small number m of mutants is introduced into the population. With $e + m$ individuals accounted for in the population $g + 1$, $p - e - m$ additional individuals need to be generated to complete the p individuals that make up population $g + 1$. This is done by producing $p - e - m$ offspring solutions through the process of *mating* or *crossover*.

A *biased random-key genetic algorithm* (Gonçalves & Resende (2011)), or simply *BRKGA*, differs from an *RKGA* in the way parents are selected for mating. While in the *RKGA* of Bean (1994) both parents are selected at random from the entire current population, in a *BRKGA* each element is generated combining a parent selected at random from the elite partition of the current population and one from the rest of the population. An individual can produce

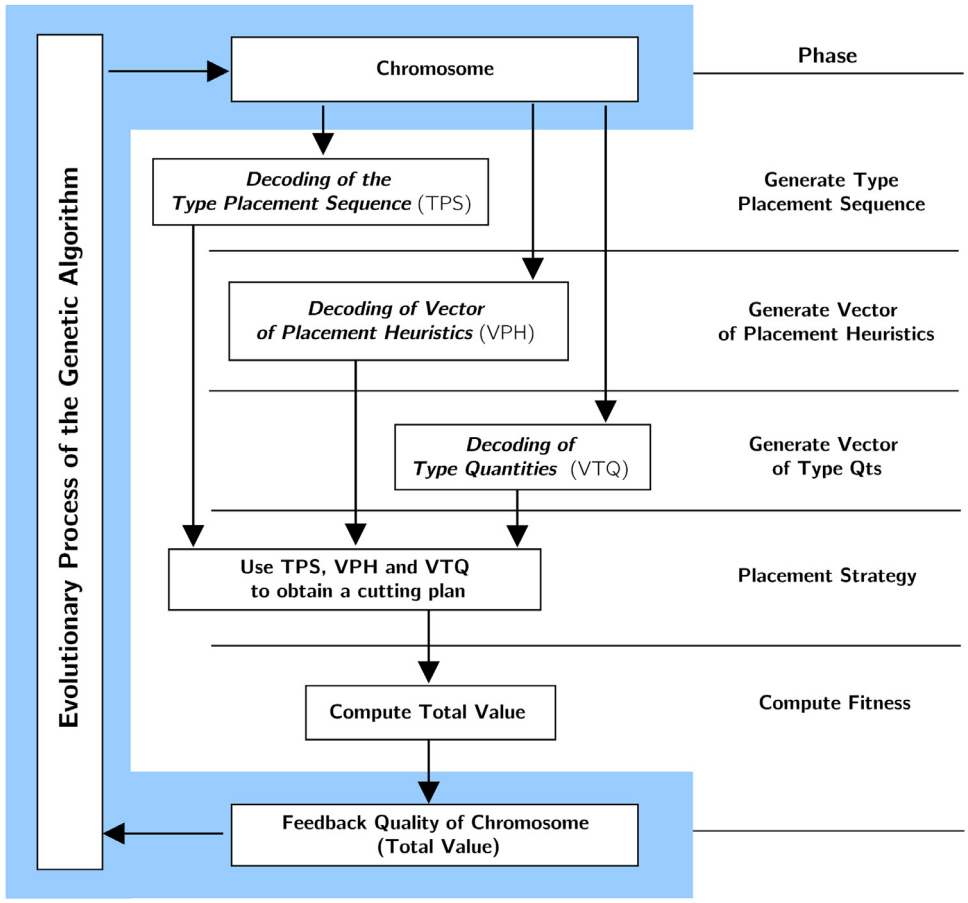


Fig. 2. Sequence of steps applied by the BRKGA to the chromosomes.

more than one offspring in the same generation since repetition in the selection of a mate is allowed. As in an *RKGA*, *parametrized uniform crossover* (Spears & Dejong, 1991) is used to implement mating in a *BRKGA*. Let ρ_e be the probability that an offspring inherits the vector component of its elite parent. Recall that r denotes the number of components in the solution vector of an individual. For $i = 1, \ldots, r$, the i-th component $c(i)$ of the offspring vector c takes on the value of the i-th component $t(i)$ of the elite parent t with probability ρ_e and the value of the i-th component $\bar{t}(i)$ of the non-elite parent $\bar{t}$ with probability $1 - \rho_e$.

When the next population is complete, i.e. when it has p individuals, fitness values are computed for all of the newly created random-key vectors and the population is partitioned into elite and non-elite individuals to start a new generation.

A *BRKGA* searches the solution space of the combinatorial optimization problem indirectly by searching the continuous r-dimensional hypercube, using the decoder to map solutions in the hypercube to solutions in the solution space of the combinatorial optimization problem where the fitness is evaluated.

To specify a biased random-key genetic algorithm, we simply need to specify its parameters, how solutions are encoded and decoded, and how their corresponding fitness values are computed. We specify our algorithm next by first showing how the cutting plan solutions are encoded and then decoded and how their fitness evaluation is computed.

5.2.1. Solution encoding and decoding

A solution to the problem is encoded as a vector of random keys, the *BRKGA* chromosome. A chromosome may represent directly a solution of the original problem or be an indirect representation. An indirect representation needs special procedures to obtain a solution. In this paper the solutions will be represented indirectly by parameters that are later used by a decoding procedure to obtain a solution. To obtain the phenotype we use the decoding procedures described in Section 5.3.4.

Each solution chromosome is made of $3N$ genes as depicted in Fig. 3, where N is, as defined above, the maximum number of generic small items that can be assigned to the large object. The first N genes are used to obtain the *Type Placement Sequence* (*TPS*), genes $N+1$ to $2N$ are used to obtain the *Vector of Placement Heuristics* (*VPH*), and genes $2N+1$ to $3N$ are used to obtain the Vector of Type Quantities, *VTQ*. The placement strategy described in Section 5.3.4 makes use of *TPS*, *VPH*, and *VTQ* to decode the chromosome into a solution (i.e. a cutting plan).

The decoding (mapping) of the first N genes of each chromosome into a *TPS* is accomplished using the following expression:

$$TPS_i = \lceil gene_i \times T \rceil \qquad i = 1, \ldots, N,$$

where $\lceil x \rceil$ corresponds to the smallest integer larger than or equal to x.

In the placement strategy we make use of two placement heuristics; *BL* (bottom-left) and *LB* (Left-Bottom) (see Section 5.3). To decide which placement heuristic to be uses for placing each small item type we will use *VPH*. The decoding of *VPH*, for $i = 1, \ldots, N$, is accomplished using the genes $N+1$ to $2N$ of each chromosome and the following expression:

$$VPH_i = \begin{cases} BL & \text{if } gene_{N+i} \leq \frac{1}{2}, \\ LB & \text{if } \frac{1}{2} < gene_{N+i}. \end{cases}$$

When placing a small item using any of the placement heuristics we try to place layers of items consisting of one or more

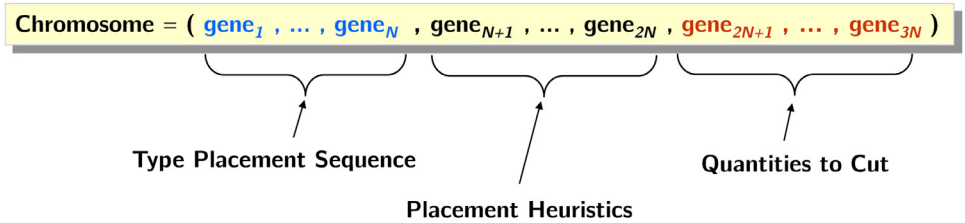


Fig. 3. Solution encoding.

small items. The number of items to be included in each layer is determined by the maximum number of items that fit in the *EMS* chosen by the placement heuristic, and *VTQ* (see Section 5.3). The decoding of *VTQ* is obtained by using genes $2N+1$ to $3N$ of each chromosome and the following expression:

$$VTQ_i = gene_i \qquad i = 1, \ldots, N.$$

A solution, i.e., a layout of small items on the large object, is constructed by the placement strategy using *TPS*, *VPH*, and *VTQ*.

5.2.2. Fitness function

To evolve the solutions, the evolutionary process needs a measure of the solution fitness, or quality. A natural fitness function for this type of problem is the value of the cutting plan defined by Eq. (1)

$$Value = \sum_{i=1}^{N} \sum_{t=1}^{T} V^t I_i^t.$$

5.3. Placement strategy

In the next sections we describe the main components of the placement strategy.

5.3.1. Maximal-spaces and the difference process

In order to place the small items on the large object we use a list S of empty maximal spaces (*EMSs*), i.e., a list of the largest empty rectangular spaces available on the large object. An empty maximal-space s is represented by its vertices with minimum and maximum coordinates (min_x^s, min_y^s and max_x^s, max_y^s, respectively). When searching for a place for positioning a small item, we need to consider only the available *EMSs* where the small item currently being considered fits. The no overlap requirement between small items will be guaranteed if we always proceed this way. We make use of the *difference process* (*DP*), developed by Lai and Chan (1997b) for generating and keeping track of the *EMSs*. Fig. 4 depicts an example of the application of the *DP* process. In the example, we assume that we have two small items (1 and 2) to be placed on the large object. Initially, since the large object is empty, we only have one empty maximal-space EMS_0 which corresponds to the entire large object (see Fig. 4(a)). After placing small item 1 in EMS_0 we update the list S of available empty maximal-spaces, Fig. 4(b) shows the four newly generated *EMSs*. We proceed by placing small item 2 in EMS_4 and the application of the*DP* process results in the six *EMSs* shown in Fig. 4(c). Every time a small item is placed on the large object, we reapply the *DP* process to update list S before we try to place the next small item.

5.3.2. Incorporating defects

Defects are incorporated in the approach by introducing a preprocessing step. It uses the difference process to create a set of maximal-spaces which exclude the defective areas and will be considered as the set of initial maximal-spaces available for cutting the small items. In Fig. 5(a) we present an example where the large object has two defects, **d1** and **d2**. After applying the difference process to the defective areas **d1** and **d2** the process finishes with the eight initial maximal-spaces depicted in Fig. 5(b). Note that the defective areas **d1** and **d2** are outside of all the eight initial maximal-spaces.

Since the defective areas might render the placement of some of the small item types infeasible, in this pre-processing step we include also a procedure that removes, from the available small types, all the infeasible types for a particular problem instance. For the example in Fig. 5 any small item type not fitting inside one of the maximal-spaces $EMS_1 - EMS_8$ will be excluded from the solution process (i.e. search space). Note also that if no type fits in the initial set of *EMSs*, the total value of the solution will be zero.

In theory the number of *EMSs* generated by the defects may increase exponentially. However, from a practical point of view, the number of defects and the number of rectangles to represent them will be small. Furthermore, in order to reduce the number of EMSs generated by the DP process, we added an extra step that checks if a newly generated EMS can fit any of the possible small items types. If the new EMS cannot fit any small item type then it is removed.

5.3.3. Placement heuristics and layers

In order to choose an *EMS* where a small item type can be placed, we use two placement heuristics: Bottom-Left (*BL*), and Left-Bottom (*LB*). The *BL* heuristic chooses the *EMS* that is closer to the bottom of the large object and then closer to the left while the *LB* heuristic chooses the *EMS* that is closer to the left of the large object and then closer to the bottom. We decided to use these two heuristics to mitigate the fact that there may be some solutions that cannot be constructed using only one of the heuristics alone, as noticed by Liu and Teng (1999).

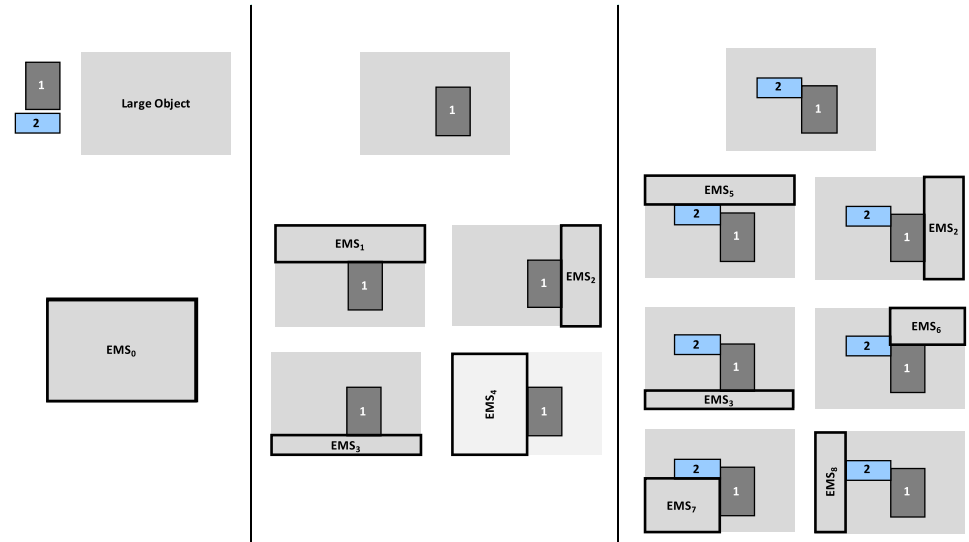
When placing a small item type in a *EMS* we try to build a layer containing several small items of the same type. We use two types of layers: horizontal and vertical. When using the *BL* heuristic, we try to create a horizontal layer and when using the *LB* heuristic, we try to create a vertical layer. The number of small items to be included in the layer, Q_i depends on the maximum number of items that fit horizontally or vertically, *maxFit*, and on VTQ_i which is generated by the BRKGA chromosome according to the following expression:

$$Q_i = \lceil VTQ_i \times maxFit \rceil \qquad i = 1, \ldots, N.$$

Fig. 6 depicts an example where item 1 has to be placed and there are already four items placed (2, 3, 4, and 5) and we will assume that $VTQ_5 = 0.45$. If we try placing item 1 using the *BL* heuristic then item 1 will be placed left of item 5. In this case $maxFit = 2$ since the maximum number of items of type 1 that can be placed in a horizontal layer is 2 and $Q_5 = \lceil 0.45 \times 2 \rceil = 1$, (see Fig. 6(a)). Alternatively, if we try placing item 1 using the *LB* heuristic then item 1 will be placed to above of item 3. In this case $maxFit = 3$, since the maximum number of items of type 1 that can be placed in a vertical layer is 3, and $Q_5 = \lceil 0.45 \times 3 \rceil = 2$, (see Fig. 6(b)).

5.3.4. Placement procedure

The placement procedure follows a sequential process which tries to place a single item at each stage i. The procedure uses *TPS*,

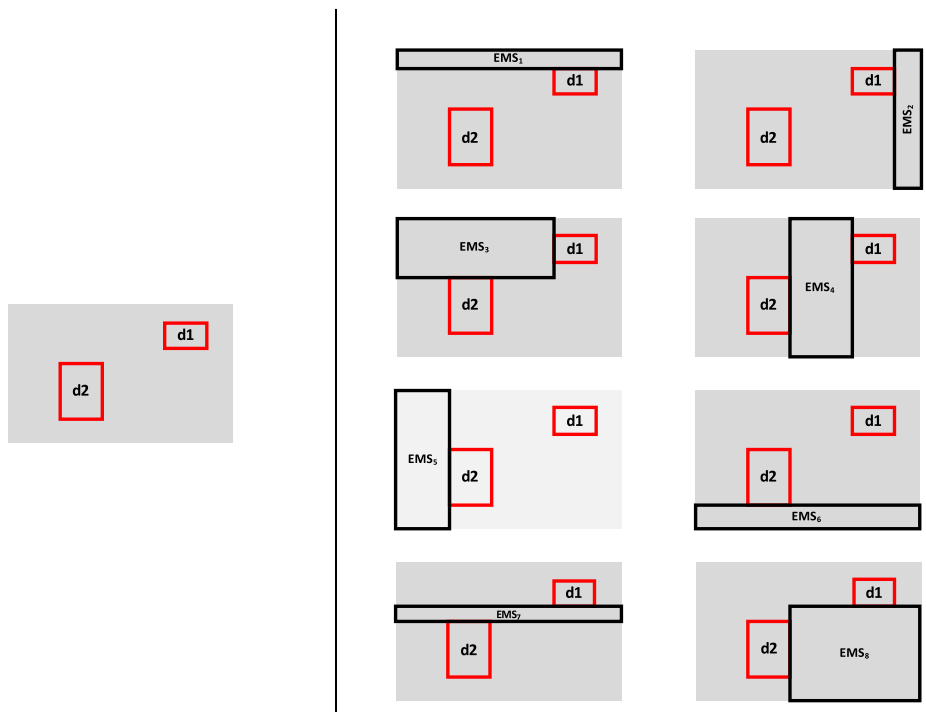


a) Large object and items 1 and 2 to be cut, and the initial empty maximal space.

b) Empty maximal-spaces after cutting item 1.

c) Empty maximal-spaces after cutting item 1 and 2.

Fig. 4. Difference process (*DP*) example.



a) Large object with two defective areas $\mathbf{d_1}$ and $\mathbf{d_2}$.

b) Empty maximal-spaces after removing the defective areas $\mathbf{d_1}$ and $\mathbf{d_2}$.

Fig. 5. Incorporating defects – example.

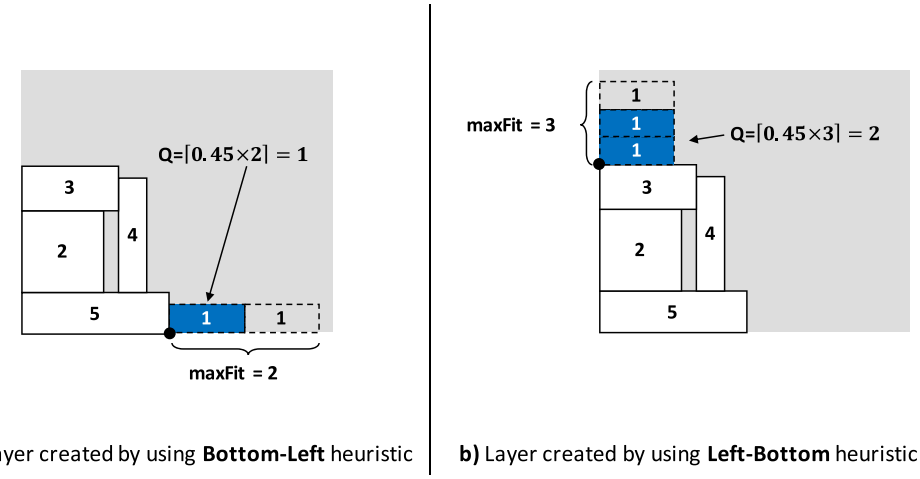
VPH, and *VTQ* evolved by the *BRKGA* to construct a cutting plan. Each stage is comprised of three main steps:

1. Type selection;
2. Layer placement;
3. State information update.

The pseudocode of the placement procedure is given in Fig. 7. The type selection at stage *i* chooses for placement the item type in the *i*th position of the *TPS* (line 4 of the pseudocode). The layer placement is carried out in line 5–18. The state information update is the final step (line 19 of the pseudocode) and consists of updating the list of empty maximal spaces, according to the layer being placed and the corresponding coordinates, using the *DP* procedure.

6. Numerical experiments

In order to evaluate the performance and the capabilities of our *BRKGA* approach, we performed a series of computational experiments. The numerical experiments were conducted on a computer with a Intel E5-2630 @2.30gigahertz CPU and 16 gigabyte of



a) Layer created by using **Bottom-Left** heuristic

b) Layer created by using **Left-Bottom** heuristic

Fig. 6. Example of *BL* and *LB* layers.

Table 1
Range of parameters for the evolutionary strategy.

Parameter	Interval
e	$(0.10 - 0.25) \times p$
m	$(0.15 - 0.30) \times p$
crossover probability (ρ_e)	$0.70 - 0.85$

Table 2
Parameter configuration for the *BRKGA* algorithm.

Parameter	Value
p	$100 \times N$
e	$min(0.25 \times p,\ 200)$
m	$0.15 \times p$
ρ_e	0.85
fitness	Total value given by Eq. (1) (to be maximized)
stopping criterion	300 generations

physical memory running the Linux operating system with Fedora release 24. The *BRKGA* was coded using the C++ programming language and the MIP models were solved using GUROBI OPTIMIZER version 6.5.2.

The next subsections report the details of the experiments and the results obtained by the approach proposed in this paper.

6.1. BRKGA configuration

The configuration of the parameters of the *BRKGA* was based on our past experience with genetic algorithms based on the same evolutionary strategy (see Gonçalves and Almeida (2002), Gonçalves and Resende (2004), Gonçalves, Mendes, and Resende (2005), Gonçalves, Mendes, and Resende (2009), Gonçalves, Resende, and Mendes (2011), Gonçalves and Sousa (2011), Gonçalves and Resende (2012), Fontes and Gonçalves (2013), Gonçalves and Resende (2013), Gonçalves, Resende, and Toso (2014), Gonçalves and Resende (2015), Ramos, Oliveira, Gonçalves, and Lopes (2015), Ramos, Oliveira, Gonçalves, and Lopes (2016), and Gonçalves, Resende, and Costa (2016) that has shown that good results can be obtained with the values of e, m, and crossover probability (ρ_e) shown in Table 1.

For the population size we obtained good results by indexing it to the size of the problem, i.e., we use small size populations for small problems and larger populations for larger problems. The configuration shown in Table 2 was held constant for all problem instances in the experiments. The experimental results demonstrate that this configuration not only provides high-quality solutions but that it is also very robust.

Table 3
Data of small item types.

t	W^t	H^t	V^t
1	40	30	10
2	68	26	12
3	50	20	8
4	60	35	18
5	45	22	9

6.2. Datasets and benchmark approaches

Since there are no benchmark instances for the problem with defects that we discuss in this paper we used existing benchmark instances for the guillotine version. We evaluate the performance of the *BRKGA* approach against other approaches and used two sets of instances (Set #1 and Set #2). Set #1 consists of 14 instances where instances 1–8 are from Carnieri et al. (1993) with only one defect and instances 9–14 are from Vianna and Arenales (2006) with 2–5 defects. The large object dimensions are $W = 200$ and $H = 100$ for all the 14 instances. The number of small item types (T) is equal to 5. The dimensions and values of the small item types are equal for all instances and are given in Table 3 (note that the values of the small item types are not proportional to their areas). The defective areas, for each instance, are represented by its vertices with minimum and maximum coordinates and are given in Table 4.

Set #2 consists of 5400 random instances generated by Afsharian et al. (2014) using an adaptation of the problem generator of Neidlein and Wäscher (2008). The instances were obtained by specifying the following problem descriptors: (1) dimensions of the large object, (2) number of small item types, (3) value of the small item types, (4) dimensions of the small item types, (5) number of defects, (6) dimensions of the defects, and (7) location of the defects.

The size of the large object was divided into the following three categories:

Category #1 – $(W, H) = (75, 75)$ and $(W, H) = (112, 50)$;
Category #2 – $(W, H) = (150, 150)$ and $(W, H) = (225, 100)$;
Category #3 – $(W, H) = (300, 300)$ and $(W, H) = (450, 200)$.

The number of small item types (T) was set to 5, 10, 15, 20, and 25 for all the categories. The values of the small item types

```
procedure PLACEMENT (TPS, VPH, VTQ)
    Let S be the set of EMSs available;

    // ** Initialization
    S ← {Large Object minus Defective Areas} // see section 5.3.2

    for i = 1, ..., N do

        // ** Type selection
        TypeSelected ← TPS_i // Select the type in the i^th position of TPS;

        // ** Layer placement
            EMS* ← ∅
            if VPH_i = BL then
                Let EMS* be the EMS, in the list of available EMSs,
                    in which a small item of type i is placed when
                    the Bottom-Left heuristic is applied;
                if EMS* ≠ ∅ then   //  an EMS was found;
                    Determine maxFit for the horizontal layer in EMS*;
                    Q_i = ⌈VTQ_i × maxFit⌉ // Compute layer quantity;
                    Place a horizontal layer with Q_i items of type TypeSelected
                        at the bottom-left corner of EMS*;
                end if
            else if VPH_i = LB then
                Let EMS* be the EMS, in the list of available EMSs,
                    in which a small item of type i is placed when
                    the Left-Bottom heuristic is applied;
                if EMS* > ∅ then   //  an EMS was found;
                    Determine maxFit for the horizontal layer in EMS*;
                    Q_i = ⌈VTQ_i × maxFit⌉ // Compute layer quantity;
                    Place a vertical layer with Q_i items of type TypeSelected
                        at the bottom-left corner of EMS*;
                end if
            end if

        // ** State information update
        Update the list of S of using
                the DP procedure of Lai & Chan (1997b);
} \end{center}
end for
end PLACEMENT;
```

Fig. 7. Pseudocode for the PLACEMENT procedure.

are equal to their area. The width and the height of the small item types are chosen uniformly from the interval $[W/\rho,\ 3W/4]$ and $[H/\rho,\ 3H/4]$, respectively, in which $\rho = 6,\ 8,\ 10$ for all the categories. The number of defects (D) is set to 1, 2, 3, and 4 for all the categories. The width and height of the defects are chosen uniformly from the interval $[W/10,\ W/6]$ and $[H/10,\ H/6]$, respectively. The location of each defect, represented by the position of the lower left corner of the defect on the large object, is generated randomly using a uniform distribution in the intervals $\left[0, W - W^d\right]$ and $\left[0, H - H^d\right]$ and are then rounded to obtain integer numbers. For each combination of large object size (3×2), number of small item types (5), size of the small item types (3), and number of defects (4), 15 instances are generated. Therefore, a total of 5400 ($3 \times 2 \times 5 \times 3 \times 4 \times 15$) problem instances are included in the numerical experiments.

We compare the results of *BRKGA* to those obtained by best solution of the 2*DMIP* model presented above, when solved by GUROBI 6.5.2 during 24 hours, 2*DMIP_24h*. Since we should at least be able to match the quality of the solutions using a guilottine approach we are also comparing it with the following two approaches recently proposed in Afsharian (2013) and Afsharian et al. (2014), and which present the best computational results to date for solutions restricted to guillotine cuts:

- *EDP* – Exact dynamic programming algorithm;
- *ADP* – Approximate dynamic programming algorithm.

Table 4
Data of defective areas.

Instance	Defective areas				*Instance*	Defective areas			
	min_x	min_y	max_x	max_y		min_x	min_y	max_x	max_y
1	100	50	105	54	10	95	0	105	10
						95	90	105	100
2	100	40	105	44		0	45	10	55
						190	45	200	55
3	100	60	105	66					
					11	95	0	105	10
4	125	20	132	30		95	90	105	100
						0	45	10	55
5	125	71	132	79		190	45	200	55
						120	60	125	65
6	30	30	40	40					
					12	0	0	5	5
7	80	40	110	58		30	15	33	19
						60	47	66	50
8	80	40	118	58		198	45	200	55
9	0	0	5	5	13	15	10	20	15
	0	95	5	10		15	85	20	90
	194	0	200	10		185	10	190	15
	194	90	200	100		185	85	190	90
					14	0	45	10	55
						190	45	200	55

Table 5
Results for Set #1.

#Inst	*BRKGA*		*2DMIP_24h*	*EDP*		*ADP*	
	TV	*time(seconds)*	*TV*	*TV*	*time(seconds)*	*TV*	*time(seconds)*
1	166	15.35	162	166	1561.10	166	18.86
2	166	15.00	163	166	1212.64	166	16.43
3	166	15.65	162	166	1212.97	166	16.47
4	164	15.32	160	164	997.12	164	18.25
5	164	15.76	160	164	999.59	164	76.92
6	164	15.08	162	164	794.55	164	0.90
7	**160**	15.09	156	158	1638.21	158	0.81
8	**156**	14.98	152	154	1692.51	154	1.21
9	**162**	16.77	152	160	181.91	160	14.32
10	158	17.59	148	158	1263.85	158	2.22
11	151	17.65	146	151	1778.15	151	26.78
12	**157**	16.21	153	156	1262.65	156	1126.44
13	150	16.28	148	150	686.78	150	9.06
14	**163**	16.07	160	160	519.33	160	1.00
average =		15.91			1128.67		94.98

EDP and ADP were programmed in C#, Version 4.0 and the experiments were been carried out on a PC with an AMD E-350@ processor with 4 gigabyte of RAM and a clock speed of 1.60 gigahertz under the Windows Operating System.

6.3. Experimental results

In this section we report on the best value and the average times obtained over 3 runs of *BRKGA* for all the datasets. For the other approaches we also report the best cost. Even though the computational times reported by the other approaches might not be comparable we report them if they are available. Since the final best dual bounds provided by GUROBI for every instance are very large and do not provide any useful information we are not including them in the result tables.

6.3.1. Results Set #1

In this section we evaluate the performance of the *BRKGA* in comparison to *2DMIP_24h*, *EDP*, and *ADP* on Set #1. As can be observed in Table 5, the *BRKGA* obtained always the best results and improved the best solution for instances 7, 8, 9, 12, and 14 (bold values in the table), using an average time of 15.91 seconds. Fig. 8 presents the solutions obtained by the *BRKGA* for all 14 instances and they all have equal or better value than the ones obtained by the more restricted approaches.

6.3.2. Results Set #2

In this section we evaluate the performance of the *BRKGA* on the Set #2, against *EDP*, *ADP*, and *2DMIP_24h*. In Tables 6 and 7 we evaluate the performance of *BRKGA* against *EDP*, and *ADP* on the Set #2 Category 1. In Tables 8–11 we evaluate the performance of *BRKGA* against only *ADP* and for Set #2 Category 2 and Category 3 since, due to the high computing times of *EDP*, Afsharian (2013) only provide results for *ADP* in Categories 2 and 3.

TV represents the average total value and *%Imp BRKGA* represents the percentage improvement of *BRKGA* over the other approaches. As can be seen in column *%Imp BRKGA*, *BRKGA* finds solutions which are, on average, better than *EDP* by 0.45% and 0.51% respectively for *Category* 1 – (75, 75) and *Category* 1 – (125, 50). The average values of *%Imp BRKGA* in relation to *ADP* are 0.88% and 1.20% respectively for *Category* 1 – (75, 75) and *Category* 1 – (125, 50). For *Category* 2 – (150, 150), *Category* 2 – (225, 100), *Category* 3 – (300, 300), *Category* 3 – (450, 200) the average values of *%Imp BRKGA* in relation to *ADP* are 1.26%, 1.52%, 1.48%, and 1.81% respectively. The average computing times of *BRKGA* are approximately 4 times smaller than *EDP* for Category 1, approximately

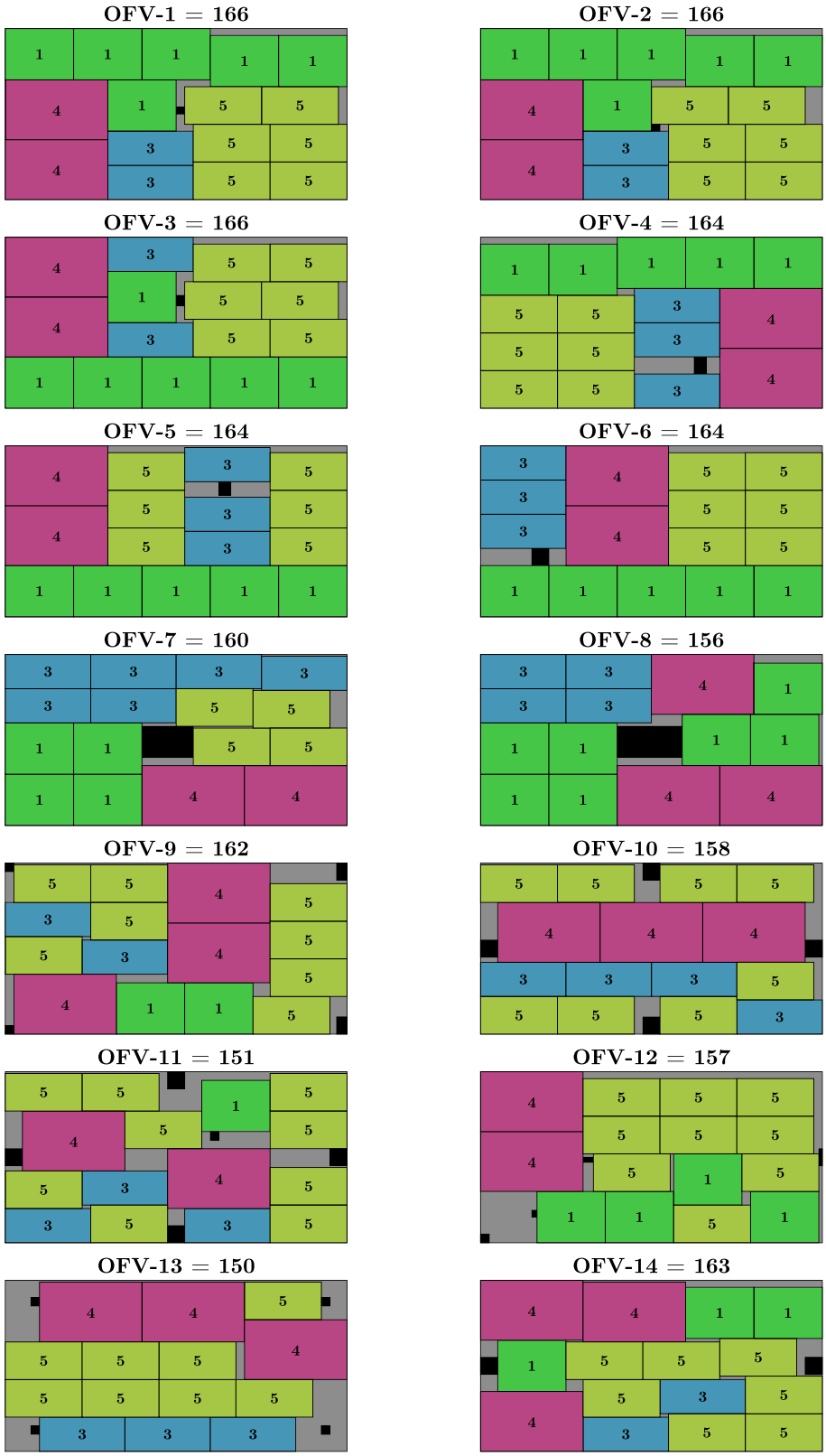


Fig. 8. Solutions obtained by BRKGA for the 14 instances in Set #1.

4 times smaller than *ADP* for Category 2, and approximately 36 times smaller than *ADP* for Category 3. In summary the results demonstrate that *BRKGA* is faster than *EDP* or *ADP* and produces always better solutions.

Next we report on the evaluation of *BRKGA* against 2*DMIP_24h*. Since solving the 2*DMIP_24h* model with GUROBI for all the 5400 instances would take 5400 days on a single computer we decided to use only a reduced set of instances from Set #2. The reduced set includes only instances with 4 defects (which will be the more difficult instances) and we have chosen randomly only one of the 15 instances of each sub-category, instance 15 was selected. This reduction resulted in a set of only 90 instances.

Table 6
Results for Category 1 – (75,75).

ρ	T	*BRKGA*		*EDP*		*ADP*		*%Imp BRKGA*	
		TV	*time (seconds)*	*TV*	*time (seconds)*	*TV*	*time (seconds)*	*vs. EDP*	*vs. ADP*
6	5	**3697.87**	6.04	3694.31	72.65	3694.31	3.35	0.10	0.10
	10	**4277.03**	10.58	4256.01	59.95	4229.85	4.52	0.49	1.12
	15	**4576.88**	14.56	4566.10	62.36	4540.38	5.36	0.24	0.80
	20	**4634.48**	18.78	4615.43	60.27	4593.41	5.76	0.41	0.89
	25	**4726.58**	21.81	4694.01	63.02	4674.66	6.19	0.69	1.11
8	5	**3701.70**	6.88	3683.23	74.84	3638.03	2.86	0.50	1.75
	10	**4411.05**	12.66	4386.16	61.88	4375.50	4.74	0.57	0.81
	15	**4641.70**	16.72	4613.23	61.83	4585.26	6.09	0.62	1.23
	20	**4925.37**	21.82	4883.78	62.82	4876.38	6.27	0.85	1.00
	25	**4862.13**	24.54	4835.60	62.30	4826.93	7.76	0.55	0.73
10	5	**4097.03**	9.19	4083.56	64.11	4055.10	3.18	0.33	1.03
	10	**4722.27**	16.45	4710.91	61.27	4686.56	5.69	0.24	0.76
	15	**4867.80**	21.60	4845.16	61.01	4826.68	6.74	0.47	0.85
	20	**4946.43**	25.01	4928.65	62.05	4920.95	9.35	0.36	0.52
	25	**4987.70**	28.97	4968.50	64.54	4960.23	10.17	0.39	0.55
average			**17.04**		**63.66**		**5.87**	**0.45**	**0.88**

Table 7
Results for Category 1 – (112,50).

ρ	T	*BRKGA*		*EDP*		*ADP*		*%Imp BRKGA*	
		TV	*time (seconds)*	*TV*	*time (seconds)*	*TV*	*time (seconds)*	*vs. EDP*	*vs. ADP*
6	5	**3548.87**	5.81	3530.41	68.98	3480.63	2.61	0.52	1.96
	10	**4348.62**	10.76	4339.18	62.18	4305.43	4.31	0.22	1.00
	15	**4512.40**	14.61	4495.20	72.47	4470.76	4.31	0.38	0.93
	20	**4642.50**	18.99	4618.16	70.28	4607.75	5.80	0.53	0.75
	25	**4715.23**	22.43	4686.21	73.42	4679.73	6.78	0.62	0.76
8	5	**3859.63**	7.71	3829.66	75.35	3752.31	2.03	0.78	2.86
	10	**4562.03**	13.06	4536.90	73.64	4523.00	4.56	0.55	0.86
	15	**4829.83**	19.43	4792.70	67.53	4766.90	5.65	0.77	1.32
	20	**4722.65**	21.41	4706.16	71.61	4685.91	5.92	0.35	0.78
	25	**4811.43**	24.58	4791.00	70.75	4772.01	8.09	0.43	0.83
10	5	**4151.63**	8.80	4138.25	69.00	4076.48	3.63	0.32	1.84
	10	**4411.72**	13.13	4396.10	73.04	4357.55	4.53	0.36	1.24
	15	**4730.77**	19.20	4688.12	72.31	4661.92	5.70	0.91	1.48
	20	**4925.95**	23.92	4895.85	69.79	4891.43	8.71	0.61	0.71
	25	**5005.03**	28.40	4988.35	70.51	4975.15	9.31	0.33	0.60
average			**16.82**		**70.72**		**5.46**	**0.51**	**1.20**

Table 8
Results for Category 2 – (150,150).

ρ	T	*BRKGA*		*ADP*		*%Imp BRKGA*
		TV	*time (seconds)*	*TV*	*time (seconds)*	*vs. ADP*
6	5	**14910.83**	6.73	14546.66	24.723	2.50
	10	**15832.06**	9.78	15544.76	35.661	1.85
	15	**17386.47**	13.17	17263.78	35.827	0.71
	20	**18383.32**	17.82	18128.60	58.636	1.41
	25	**19000.23**	22.51	18716.65	69.199	1.52
8	5	**14429.35**	6.63	14276.08	27.278	1.07
	10	**17855.42**	13.92	17632.83	40.890	1.26
	15	**18851.20**	17.97	18615.33	72.122	1.27
	20	**19425.53**	22.76	19193.25	73.607	1.21
	25	**19619.03**	26.51	19435.88	131.198	0.94
10	5	**15244.85**	7.62	15067.86	24.038	1.17
	10	**18939.72**	16.30	18775.98	63.492	0.87
	15	**18911.88**	19.68	18685.03	71.939	1.21
	20	**19575.43**	22.48	19372.35	99.747	1.05
	25	**19900.18**	30.80	19724.56	137.128	0.89
average			**16.98**		**64.37**	**1.26**

Tables 12 and 14 present the performance of *BRKGA* against 2*DMIP_24h* model on GUROBI 6.5.2 on the reduced set of instances of Set #2. The analysis of the tables shows that *BRKGA* generated, for all the 90 instances, a solution with better total value, *TV*, spending only an average of 13.73, 18.35, and 20.30 seconds, respectively, for ρ equal to 6, 8, and, 10. The average percentage improvement of *BRKGA* over 2*DMIP_24h*, *%Imp BRKGA*, is 1.39%, 3.04%, and 4.46% for ρ, equal to 6, 8, and 10. *BRKGA* obtained better *TV* in 50 out of the 90 instances and the maximum *%Imp BRKGA* was 19.70% and occurred for instance C150x150-15-10-15-4 (row 13 in Table 14).

Overall, for the test instances used, omitting the guillotine cut constraint improves the solutions only by a small margin. However, because of increased market competition, any small

Table 9
Results for Category 2 – (225,100).

ρ	T	BRKGA		ADP		%Imp BRKGA
		TV	time (seconds)	TV	time (seconds)	vs. ADP
6	5	**13636.08**	5.97	13420.90	21.29	1.60
	10	**17022.84**	10.49	16709.06	43.03	1.88
	15	**17290.72**	13.03	17058.16	45.42	1.36
	20	**18740.97**	17.38	18475.80	62.96	1.44
	25	**18815.77**	22.04	18591.30	74.73	1.21
8	5	**15222.01**	6.75	15024.26	23.33	1.32
	10	**17845.32**	13.01	17541.91	53.70	1.73
	15	**18735.07**	16.32	18491.06	63.04	1.32
	20	**19104.65**	22.00	18814.46	84.30	1.54
	25	**19500.98**	25.90	19215.23	109.88	1.49
10	5	**15634.70**	7.76	15248.20	35.21	2.53
	10	**17503.34**	13.39	17191.66	47.83	1.81
	15	**19078.07**	18.57	18733.61	92.64	1.84
	20	**19688.90**	25.24	19537.85	131.77	0.77
	25	**19810.03**	27.57	19561.21	122.08	1.27
average			**16.36**		**67.41**	**1.52**

Table 10
Results for Category 3 – (300,300).

ρ	T	BRKGA		ADP		%Imp BRKGA
		TV	time (seconds)	TV	time (seconds)	vs. ADP
6	5	**60110.69**	6.48	58707.60	123.68	2.39
	10	**65872.74**	9.68	64857.25	225.89	1.57
	15	**69915.07**	13.32	68832.68	265.06	1.57
	20	**72935.98**	17.90	71708.11	501.14	1.71
	25	**75630.38**	22.09	74345.13	505.45	1.73
8	5	**57690.52**	6.11	56809.91	96.13	1.55
	10	**69352.93**	12.43	68244.58	311.65	1.62
	15	**74210.50**	17.27	73047.51	529.24	1.59
	20	**76016.22**	21.72	74767.43	819.22	1.67
	25	**77620.77**	26.00	76713.05	1422.33	1.18
10	5	**60054.53**	7.25	59584.45	150.97	0.79
	10	**73493.90**	14.32	72717.70	398.99	1.07
	15	**74343.53**	18.71	73389.63	577.09	1.30
	20	**79091.05**	26.72	78297.61	1545.06	1.01
	25	**79331.70**	30.33	78211.60	1558.21	1.43
average			**16.69**		**602.01**	**1.48**

Table 11
Results for Category 3 – (450,200).

ρ	T	BRKGA		ADP		%Imp BRKGA
		TV	time (seconds)	TV	time (seconds)	vs. ADP
6	5	**55585.71**	5.81	54692.40	122.87	1.63
	10	**68363.29**	10.56	67097.40	297.51	1.89
	15	**72802.43**	15.14	71595.45	417.33	1.69
	20	**72508.94**	17.60	71311.08	384.57	1.68
	25	**76146.53**	23.10	75154.26	844.27	1.32
8	5	**58947.60**	6.17	57095.48	120.32	3.24
	10	**68540.91**	12.05	67306.91	330.42	1.83
	15	**73547.05**	17.23	72500.68	653.24	1.44
	20	**77835.85**	22.73	76691.43	957.87	1.49
	25	**78806.68**	26.76	77629.33	1268.14	1.52
10	5	**65034.44**	8.71	63169.15	154.15	2.95
	10	**68596.42**	13.18	67114.36	378.62	2.21
	15	**76776.17**	19.32	75415.03	849.88	1.80
	20	**78587.83**	25.98	77600.43	1098.54	1.27
	25	**78554.79**	29.16	77686.03	1509.38	1.12
average			**16.90**		**625.81**	**1.81**

improvement in the efficiency of the cutting process is relevant. The small improvements observed in the test instances are mainly due to the relative small size of the small items when compared with the large object. Fig. 9 shows an example where the large object has dimension equal to 6×6 and the small items have dimensions equal to 2×4 and can be rotated to 4×2. When a small defect is placed close to the center of the large object using a non-guillotine solution instead of guillotine solution improves the material usage by 33% (from 66.7 to 88.9%) (see Fig. 9).

7. Extension to defects with different quality areas

In some applications, e.g. cutting natural products such as wood or leather, defects are not 0–1 features, they may come in the form

Table 12
Results for *BRKGA* vs. *2DMIP_24h* ($\rho = 6$).

#	*Instance*	*BRKGA*		*2DMIP_24h*	
		TV	*time (seconds)*	*TV*	*%Imp BRKGA*
1	C75x75-05-06-15-4	2,948	7.42	2,738	7.67
2	C75x75-10-06-15-4	2,786	8.75	2,786	0.00
3	C75x75-15-06-15-4	4,446	17.29	4,392	1.23
4	C75x75-20-06-15-4	4,410	22.02	4,208	4.80
5	C75x75-25-06-15-4	4,658	24.39	4,320	7.82
6	C112x50-05-06-15-4	3,708	7.39	3,708	0.00
7	C112x50-10-06-15-4	3,770	11.07	3,770	0.00
8	C112x50-15-06-15-4	3,862	12.24	3,862	0.00
9	C112x50-20-06-15-4	3,840	17.10	3,774	1.75
10	C112x50-25-06-15-4	3,976	19.18	3,836	3.65
11	C150x150-05-06-15-4	10,742	5.91	10,742	0.00
12	C150x150-10-06-15-4	13,203	7.56	13,203	0.00
13	C150x150-15-06-15-4	17,181	13.94	17,181	0.00
14	C150x150-20-06-15-4	16,511	18.40	16,397	0.70
15	C150x150-25-06-15-4	15,958	22.47	15,748	1.33
16	C225x100-05-06-15-4	4,235	2.65	4,235	0.00
17	C225x100-10-06-15-4	16,413	12.88	16,361	0.32
18	C225x100-15-06-15-4	14,136	11.90	14,136	0.00
19	C225x100-20-06-15-4	17,404	17.24	16,720	4.09
20	C225x100-25-06-15-4	16,518	21.44	16,518	0.00
21	C300x300-05-06-15-4	33,663	4.01	33,663	0.00
22	C300x300-10-06-15-4	53,235	10.44	53,235	0.00
23	C300x300-15-06-15-4	52,910	11.54	52,910	0.00
24	C300x300-20-06-15-4	68,700	21.30	68,034	0.98
25	C300x300-25-06-15-4	62,903	21.29	62,903	0.00
26	C450x200-05-06-15-4	57,761	7.75	54,309	6.36
27	C450x200-10-06-15-4	63,954	9.67	63,954	0.00
28	C450x200-15-06-15-4	60,317	13.82	59,805	0.86
29	C450x200-20-06-15-4	51,967	12.91	51,967	0.00
30	C450x200-25-06-15-4	59,634	18.11	59,634	0.00
average			13.73		1.39

Table 13
Results for *BRKGA* vs. *2DMIP_24h* ($\rho = 8$).

#	*Instance*	*BRKGA*		*2DMIP_24h*	
		OFV	*time (seconds)*	*OFV*	*%Imp BRKGA*
1	C75x75-05-08-15-4	4,352	13.60	4,175	4.24
2	C75x75-10-08-15-4	4,182	13.17	4,182	0.00
3	C75x75-15-08-15-4	3,450	13.34	3,450	0.00
4	C75x75-20-08-15-4	4,500	25.41	4,238	6.18
5	C75x75-25-08-15-4	4,691	30.86	4,239	10.66
6	C112x50-05-08-15-4	4,125	9.09	4,125	0.00
7	C112x50-10-08-15-4	4,340	15.45	4,340	0.00
8	C112x50-15-08-15-4	4,767	22.97	4,499	5.96
9	C112x50-20-08-15-4	4,157	22.27	3,894	6.75
10	C112x50-25-08-15-4	4,545	27.46	4,125	10.18
11	C150x150-05-08-15-4	11,568	6.64	11,568	0.00
12	C150x150-10-08-15-4	17,795	21.08	16,995	4.71
13	C150x150-15-08-15-4	16,096	17.26	14,946	7.69
14	C150x150-20-08-15-4	17,814	24.33	17,225	3.42
15	C150x150-25-08-15-4	19,499	35.25	18,663	4.48
16	C225x100-05-08-15-4	14,843	8.13	14,843	0.00
17	C225x100-10-08-15-4	9,804	7.62	9,804	0.00
18	C225x100-15-08-15-4	17,421	16.67	17,421	0.00
19	C225x100-20-08-15-4	18,618	23.30	18,118	2.76
20	C225x100-25-08-15-4	16,178	23.54	16,178	0.00
21	C300x300-05-08-15-4	49,098	6.88	49,098	0.00
22	C300x300-10-08-15-4	30,806	7.47	30,806	0.00
23	C300x300-15-08-15-4	68,442	20.63	67,759	1.01
24	C300x300-20-08-15-4	64,973	20.61	64,470	0.78
25	C300x300-25-08-15-4	73,898	26.98	69,255	6.70
26	C450x200-05-08-15-4	52,368	5.71	52,368	0.00
27	C450x200-10-08-15-4	59,341	11.16	59,341	0.00
28	C450x200-15-08-15-4	73,756	20.56	69,292	6.44
29	C450x200-20-08-15-4	70,420	23.83	69,123	1.88
30	C450x200-25-08-15-4	71,909	29.34	67,037	7.27
average			18.35		3.04

Table 14
Results for *BRKGA* vs. *2DMIP_24h* ($\rho = 10$).

#	*Instance*	*BRKGA*		*2DMIP_24h*	
		OFV	*time (seconds)*	*OFV*	*%Imp BRKGA*
1	C75x75-05-10-15-4	4,730	23.24	4,552	3.91
2	C75x75-10-10-15-4	4,486	19.58	4,270	5.06
3	C75x75-15-10-15-4	4,417	20.19	4,383	0.78
4	C75x75-20-10-15-4	4,556	27.44	4,345	4.86
5	C75x75-25-10-15-4	4,493	27.58	4,348	3.33
6	C112x50-05-10-15-4	3,228	8.12	3,228	0.00
7	C112x50-10-10-15-4	3,876	12.88	3,830	1.20
8	C112x50-15-10-15-4	4,693	23.31	4,320	8.63
9	C112x50-20-10-15-4	4,339	21.65	4,291	1.12
10	C112x50-25-10-15-4	4,508	26.66	4,416	2.08
11	C150x150-05-10-15-4	16,210	10.47	16,210	0.00
12	C150x150-10-10-15-4	16,904	17.62	15,618	8.23
13	C150x150-15-10-15-4	17,938	25.57	14,986	19.70
14	C150x150-20-10-15-4	16,306	18.24	16,306	0.00
15	C150x150-25-10-15-4	18,712	32.10	17,484	7.02
16	C225x100-05-10-15-4	13,322	7.51	13,322	0.00
17	C225x100-10-10-15-4	16,232	12.65	16,232	0.00
18	C225x100-15-10-15-4	18,281	22.87	17,975	1.70
19	C225x100-20-10-15-4	17,507	22.01	16,350	7.08
20	C225x100-25-10-15-4	18,837	22.91	18,019	4.54
21	C300x300-05-10-15-4	61,548	7.88	61,548	0.00
22	C300x300-10-10-15-4	62,653	12.60	62,650	0.00
23	C300x300-15-10-15-4	58,932	11.75	58,932	0.00
24	C300x300-20-10-15-4	78,269	29.94	72,018	8.68
25	C300x300-25-10-15-4	73,632	33.67	63,584	15.80
26	C450x200-05-10-15-4	49,920	9.44	49,920	0.00
27	C450x200-10-10-15-4	65,216	15.07	65,216	0.00
28	C450x200-15-10-15-4	74,853	22.94	68,554	9.19
29	C450x200-20-10-15-4	74,314	28.72	68,501	8.49
30	C450x200-25-10-15-4	75,102	34.30	66,794	12.44
average			20.30		4.46

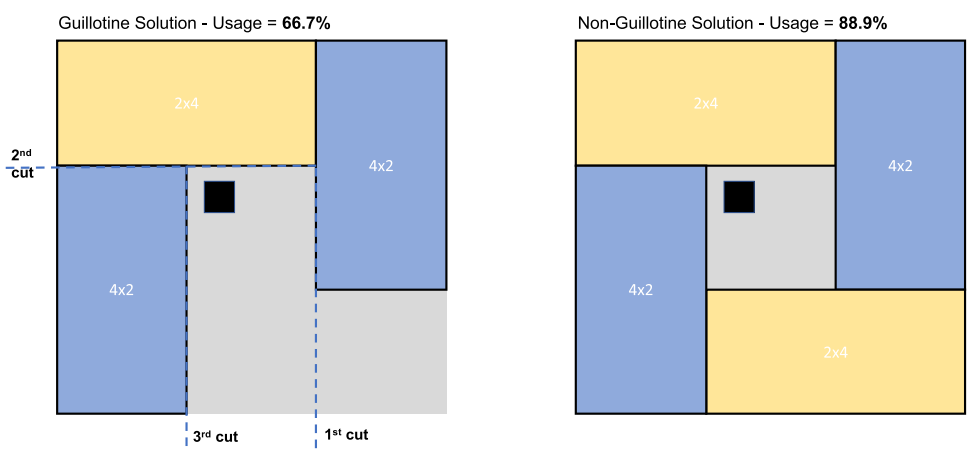


Fig. 9. Example of a large usage improvement by using a non-guillotine solution.

of different quality areas. Furthermore, each item type also has a minimum required quality level. In this section we propose a possible extension to the above MIP model and BRKGA to handle this type of problem.

In the extended MIP/BRKGA only the defects with quality below the minimum required quality level will be handled exactly as before. For the defects with quality level above or equal to the minimum required quality level, since they can be included totally or partially in any of the small items, we propose that a penalty be added to the objective function to reduce the value of the items including them. To determine if a defect e with quality level above the minimum required level is included in a small item i we add the 0–1 variable $Z_{i,e}$ and the following 4 constraints to the MIP model for every (i, e) pair.

$$x_i - X_e + W\,(1 - Z_{i,e}) \le \frac{w_i + W_e}{2} \qquad \forall\, i, e \tag{17}$$

$$X_d - x_i + W\,(1 - Z_{i,e}) \le \frac{w_i + W_e}{2} \qquad \forall\, i, e \tag{18}$$

$$y_i - Y_d + H\,(1 - Z_{i,e}) \le \frac{h_i + H_e}{2} \qquad \forall\, i, e \tag{19}$$

$$Y_d - y_i + H\,(1 - Z_{i,e}) \le \frac{h_i + H_e}{2} \qquad \forall\, i, e \tag{20}$$

Assuming that including defect e in a generic item t reduces its value by RV_e^t then the reduction in value for item i of type t will given by $rv_i^t \ge 0$ defined in by following constraint

$$RV_e^t\left(Z_{i,e} + I_i^t - 1\right) \le rv_i^t \qquad \forall\, i, t \tag{21}$$

The extended objective function of the MIP would be added the following additional term,

$$-\sum_{i=1}^{N}\sum_{t=1}^{T} rv_i^t. \tag{22}$$

The BRKGA decoder will remain exactly the same but the fitness function should be modified to account for the reduction in value of items including defects with quality level above the minimum required level.

8. Concluding remarks

In this paper we present a *MIP* model and a biased random key genetic algorithm for a two-dimensional non-guillotine cutting problem, where a fixed set of small rectangles has to be cut from a larger stock rectangle with defects so as to maximize the value of the rectangles cut. The *BRKGA* proposed hybridizes a novel placement procedure with a biased random-key genetic algorithm. The approach uses a maximal-space representation to manage the free spaces and the defects. The approach is tested on two benchmark sets having a total of 5414 instances taken from the literature and is compared against the best solution of obtained by running GUROBI 6.5.2 during 24 hours, 2*DMIP_24h*, and two existing approaches using exact (EDP) and approximate dynamic programming (ADP). *BRKGA* finds solutions which are, on average, better than EDP by 0.45% and 0.51% respectively for $Category\,1-(75,75)$ and $Category\,1-(125,50)$. The improvements in relation to ADP are 0.88% and 1.20% respectively for $Category\,1-(75,75)$ and $Category\,1-(125,50)$. For $Category\,2-(150,150)$, $Category\,2-(225,100)$, $Category\,3-(300,300)$, $Category\,3-(450,200)$ the improvement in relation to ADP are 1.26%, 1.52%, 1.48%, and 1.81% respectively *BRKGA* also obtained better total value in 50 out of the 90 instances solved by GUROBI over 24 hours using the MIP model (2*DMIP_24h*). The average percentage improvement of BRKGA over 2*DMIP_24h*, is 1.39%, 3.04%, and 4.46% respectively for ρ=6, 8,10 and the maximum was 19.70%. The average computing times of *BRKGA* are approximately 4 times smaller than EDP for *Category* 1 and ADP for *Category* 2, and approximately 36 times smaller than ADP for *Category*3.

Acknowledgments

Supported by Project “NORTE-01-0145-FEDER-000020” financed by the North Portugal Regional Operational Programme (NORTE 2020), under the PORTUGAL 2020 Partnership Agreement, and through the European Regional Development Fund (ERDF).

References

Afsharian, M. (2013). *Two-dimensional, rectangular, guillotineable-layout cutting problem with defects*. Shaker Verlag GmbH, Aachen, Germany.
Afsharian, M., Niknejad, A., & Wäscher, G. (2014). A heuristic, dynamic programming-based approach for a two-dimensional cutting problem with defects. *OR Spectrum, 36*(4), 971–999.
Alvarez-Valdes, R., Parreño, F., & Tamarit, J. (2005). A GRASP algorithm for constrained two-dimensional non-guillotine cutting problems. *Journal of Operational Research Society, 56*, 414–425.
Alvarez-Valdes, R., Parreño, F., & Tamarit, J. (2007). A tabu search algorithm for a two-dimensional non-guillotine cutting problem. *European Journal of Operational Research, 183*, 1167–1182.
Andrade, R., Birgin, E. G., Morabito, R., & Ronconi, D. P. (2014). Mip models for two-dimensional non-guillotine cutting problems with usable leftovers. *Journal of the Operational Research Society, 65*(11), 1649–1663.
Arenales, M., & Morabito, R. (1995). An AND/OR graph approach to the solution of two dimensional non-guillotine cutting problems. *European Journal of Operational Research, 84*, 599–617.
Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing, 6*, 154–160.
Beasley, J. (2004). A population heuristic for constrained two-dimensional non-guillotine cutting. *European Journal of Operational Research, 156*(3), 601–627.
Beasley, J. E. (1985). An exact two-dimensional non-guillotine cutting tree search procedure. *Operations Research, 33*, 49–64.
Boschetti, M. A., Hadjiconstantinou, E., & Mingozzi, A. (2002). New upper bounds for the two-dimensional orthogonal non-guillotine cutting stock problem. *IMA Journal of Management Mathematics, 13*, 95–119.
Carnieri, C., Mendoza, G. A., & Luppold, W. G. (1993). Optimal cutting of dimension parts from lumber with a defect: A heuristic solution procedure. *Forest Products Journal, 43*(9), 66.
Fontes, D. B. M. M., & Gonçalves, J. F. (2013). A multi-population hybrid biased random key genetic algorithm for hop-constrained trees in nonlinear cost flow networks. *Optimization Letters, 7*(6), 1303–1324. doi:10.1007/s11590-012-0505-5.
Gilmore, P. C., & Gomory, R. E. (1961). A linear programming approach to the cutting-stock problem. *Operations research, 9*(6), 849–859.
Gilmore, P. C., & Gomory, R. E. (1965). Multistage cutting stock problems of two and more dimensions. *Operations research, 13*(1), 94–120.
Goldberg, D. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
Gonçalves, J. F., & Almeida, J. (2002). A hybrid genetic algorithm for assembly line balancing. *Journal of Heuristics, 8*, 629–642.
Gonçalves, J. F., Mendes, J., & Resende, M. G. C. (2005). A hybrid genetic algorithm for the job shop scheduling problem. *European Journal of Operational Research, 167*, 77–95.
Gonçalves, J. F., Mendes, J., & Resende, M. G. C. (2009). A genetic algorithm for the resource constrained multi-project scheduling problem. *European Journal of Operational Research, 189*, 1171–1190.
Gonçalves, J. F., & Resende, M. G. C. (2011). Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics, 17*, 487–525.
Gonçalves, J. F., & Resende, M. G. C. (2012). A parallel multi-population biased random-key genetic algorithm for a container loading problem. *Computers and Operations Research, 39*, 179–190.
Gonçalves, J. F., & Resende, M. G. C. (2013). A biased random key genetic algorithm for 2d and 3d bin packing problems. *International Journal of Production Economics, 145*, 500–510.
Gonçalves, J. F., & Resende, M. G. C. (2015). A biased random-key genetic algorithm for the unequal area facility layout problem. *European Journal of Operational Research, 246*(1), 86–107.
Gonçalves, J. F., Resende, M. G. C., & Costa, M. D. (2016). A biased random-key genetic algorithm for the minimization of open stacks problem. *International Transactions in Operational Research, 23*(1–2), 25–46.
Gonçalves, J. F., Resende, M. G. C., & Mendes, J. J. M. (2011). A biased random-key genetic algorithm with forward-backward improvement for the resource constrained project scheduling problem. *Journal of Heuristics, 17*, 467–486.
Gonçalves, J. F., Resende, M. G. C., & Toso, R. F. (2014). An experimental comparison of biased and unbiased random-key genetic algorithms. *Pesquisa Operacional, 34*, 143–164.
Gonçalves, J. F., & Sousa, P. S. A. (2011). A genetic algorithm for lot sizing and scheduling under capacity constraints and allowing backorders. *International Journal of Production Research, 49*, 2683–2703.
Gonçalves, J. F., & Resende, M. G. C. (2004). An evolutionary algorithm for manufacturing cell formation. *Computers and Industrial Engineering, 47*, 247–273.
Hadjiconstantinou, E., & Christofides, N. (1995). An exact algorithm for general, orthogonal, two dimensional knapsack problems. *European Journal of Operations Research, 83*, 39–56.
Hahn, S. G. (1968). On the optimal cutting of defective sheets. *Operations Research, 16*(6), 1100–1114.
Healy, P., Creavin, M., & Kuusik, A. (1999). An optimal algorithm for rectangle placement. *Operations Research Letters, 24*, 73–80.
Herz, J. (1972). Recursive computational procedure for two-dimensional stock cutting. *IBM Journal of Research and Development, 16*(5), 462–469.
Lai, K. K., & Chan, J. W. M. (1997a). An evolutionary algorithm for the rectangular cutting stock problem. *International Journal of Industrial Engineering, 4*, 130–139.
Lai, K. K., & Chan, J. W. M. (1997b). Developing a simulated annealing algorithm for the cutting stock problem. *Computers and Industrial Engineering, 32*, 115–127.
Leung, T. W., Chan, C. K., & Troutt, M. D. (2003). Application of a mixed simulated annealing-genetic algorithm heuristic for the two-dimensional orthogonal packing problem. *European Journal of Operational Research, 145*(3), 530–542.
Liu, D., & Teng, H. (1999). An improved BL-algorithm for genetic algorithm of the orthogonal packing of rectangles. *European Journal of Operational Research, 112*, 413–420.
Neidlein, V., & Wäscher, G. (2008). SLOPPGEN: A problem generator for the two-dimensional rectangular single large object placement problem with a single defect. *Technical Report*. Otto-von-Guericke University Magdeburg, Faculty of Economics and Management.
Ramos, A. G., Oliveira, J. F., Gonçalves, J. F., & Lopes, M. P. (2015). Dynamic stability metrics for the container loading problem. *Transportation Research Part C: Emerging Technologies, 60*, 480–497.
Ramos, A. G., Oliveira, J. F., Gonçalves, J. F., & Lopes, M. P. (2016). A container loading algorithm with static mechanical equilibrium stability constraints. *Transportation Research Part B: Methodological, 91*, 565–581.
Scheithauer, G., & Terno, J. (1993). Modeling of packing problems. *Optimization, 28*, 63–84.
Spears, W. M., & Dejong, K. A. (1991). On the virtues of parameterized uniform crossover. In *Proceedings of the fourth international conference on genetic algorithms* (pp. 230–236).
Tsai, R. D., Malstrom, E. M., & Meeks, H. D. (1988). A two-dimensional palletizing procedure for warehouse loading operations. *IIE Transactions, 20*, 418–425.
Vianna, A. C. G., & Arenales, M. N. (2006). O problema de corte de placas defeituosas. *Pesquisa Operacional, 26*(2), 185–202.
Wäscher, G., Haußner, H., & Schumann, H. (2007). An improved typology of cutting and packing problems. *European Journal of Operational Research, 183*(3), 1109–1130.